

Práctica 3

Parte 1

Cuando hablamos de tiempo real, es fundamental tener conocimiento preciso de cómo se comporta el sistema que se desarrolla. Como parte del proceso de depuración, la herramienta Segger SystemView nos va a permitir capturar, almacenar y graficar los eventos que suceden en la aplicación en tiempo de ejecución. Estos eventos pueden ser expiración de temporizadores, cambios de contexto en el planificador, llamadas a funciones, mensajes entre procesos, etc.

Este análisis es fundamental para tener certezas sobre la precedencia de procesos, su duración exacta, uso de memoria, y todo lo que necesitamos para garantizar los requerimientos exigibles al tiempo real.

Segger SystemView es una aplicación gratuita para fines no comerciales, que puedes descargar para su instalación desde el siguiente enlace:

<https://www.segger.com/downloads/systemview/>

Una vez instalada la aplicación, sigue los siguientes pasos:

1. Crea el proyecto STR2024_P3 y la configuración “parte1_1”. En el fichero main.c correspondiente, copia el siguiente código:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/timers.h"
#include "esp_log.h"
#include "SEGGER_SYSVIEW.h"

static const char* TAG = "P3_1_1";

// Estructura para configurar una tarea
typedef struct task_config_t
{
    char task_name [10];
    uint16_t matrix_size;
    uint16_t iterations;
    uint16_t period; // msecs.
}task_config_t;

// Función auxiliar para introducir un procesamiento de duración variable
void matrix_multiply(uint16_t matrix_size, uint16_t iterations)
{

```

```
float matrix_A[matrix_size][matrix_size];
float matrix_B[matrix_size][matrix_size];
float result[matrix_size][matrix_size];

uint16_t i, j, k, iter;

// Rellena la matriz con valores aleatorios
ESP_LOGI(TAG, "matrix_multiply: %d x %d elementos, %d veces",
          matrix_size, matrix_size, iterations);
for (i = 0; i < matrix_size; ++i)
{
    for (j = 0; j < matrix_size; ++j)
    {
        matrix_A[i][j] = (float)rand();
        matrix_B[i][j] = (float)rand();
    }
}

// multiplicación de matrices
for (iter = 0; iter < iterations; ++iter)
{
    for (i = 0; i < matrix_size; ++i)
    {
        for (j = 0; j < matrix_size; ++j)
        {
            result[i][j] = 0.0;
            for (k = 0; k < matrix_size; ++k)
            {
                result[i][j] += matrix_A[i][k] * matrix_B[k][j];
            }
        }
    }
}

// Callback del timer de la tarea
void timer_callback(TimerHandle_t xTimer)
{
    SemaphoreHandle_t *pxSemaphore = (SemaphoreHandle_t *)pvTimerGetTimerID(xTimer);
    // libera semáforo para iteración
    xSemaphoreGiveFromISR(*pxSemaphore, pdFALSE);
}

// Tarea
void task(void *pvParameters)
{
    task_config_t* config = (task_config_t *) pvParameters;
    SemaphoreHandle_t sync_sem = xSemaphoreCreateBinary();
    char tmrname[50];
    sprintf(tmrname, "PeriodicTimer[%s]", config->task_name);
    TimerHandle_t timer = xTimerCreate("PeriodicTimer", pdMS_TO_TICKS(config->period),
                                       pdTRUE, (void *)&sync_sem, timer_callback);
```

```
xTimerStart(timer,0);

while (1)
{
    ESP_LOGI(TAG, "%s: Multiplicando matriz aleatoria...", config->task_name);
    matrix_multiply(config->matrix_size, config->iterations);
    ESP_LOGI(TAG, "%s: Multiplicación finalizada.", config->task_name);
    // bloquea a la espera del timer
    xSemaphoreTake(sync_sem, portMAX_DELAY);
}

}

void app_main()
{
    // Configura e inicia el proceso de captura de eventos de Segger SystemView
    SEGGER_SYSVIEW_Conf();
    SEGGER_SYSVIEW_Start();

    // Estructuras de datos para configurar las tareas
    task_config_t config_t1 = {"t1", 5, 1, 100}; // tarea frecuente pero rápida
    task_config_t config_t2 = {"t2", 50, 1, 500}; // tarea poco frecuente pero lenta

    // Creación de las tareas
    xTaskCreatePinnedToCore(task, "TaskA", 50*1024, (void*) &config_t1, 1, NULL, 0); // tarea 1,
core 0, prioridad 1
    xTaskCreatePinnedToCore(task, "TaskB", 50*1024, (void*) &config_t2, 1, NULL, 0); // tarea 2,
core 0, prioridad 1

    // Loop
    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

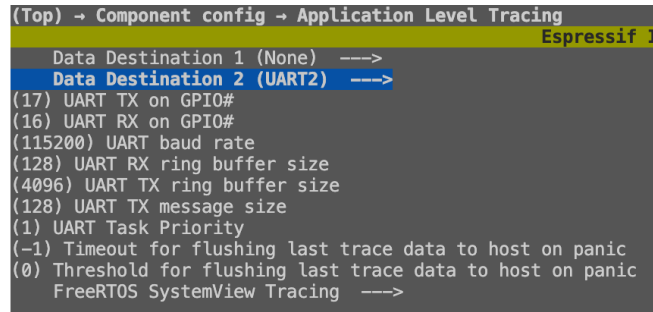
Observa cómo se utiliza de forma avanzada el puntero pvParameters de las tareas para pasar argumentos. En el ejemplo, se define una estructura para tal efecto (task_config_t). Gracias a esta estrategia, el código de la tarea es compartido por ambas, sin necesidad de replicar código ni crear variables globales; esto contribuye a tener un código más modular, reutilizable y fácil de mantener.

Observa cómo las tareas 1 y 2 se crean para que compartan el mismo core y la misma prioridad. Esto tiene sentido en el ejercicio académico de esta práctica. En un sistema en producción, interesaría que cada tarea utilizase un core diferente, para incrementar la eficiencia del sistema.

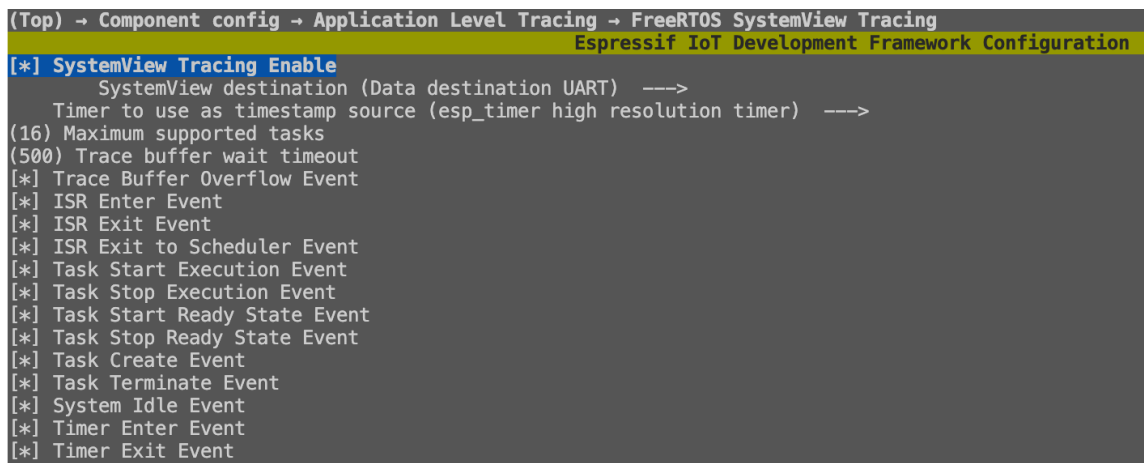
2. SEGGER SystemView captura, graba y representa las trazas que FreeRTOS exporta a través de JTAG o UART. Esta característica hay que activarla en el sistema.

a. Lanza Menuconfig para “parte1_1” y navega al menú Component Config -> Application Level Tracing.

- b. En la opción Data Destination 2, selecciona la opción UART y establece la siguiente configuración:



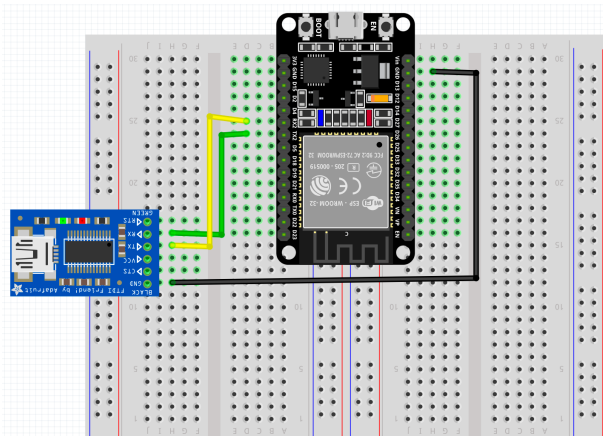
- c. Dentro del submenú FreeRTOS SystemView Tracing, establece la siguiente configuración:



- d. Sal de Menuconfig y guarda la configuración.

ESP32-WROOM-32 tiene dos UARTs (puertos serie). El 0 es el que estamos utilizando con el cable USB (está conectado a través de un chip bridge USB-UART), el segundo está mapeado en los pines 16 y 17 de la placa. Consulta el pinout disponible en Moodle para comprobarlo. En esta configuración, estamos indicando a FreeRTOS que emita las trazas a través de esa segunda UART.

3. Haz el siguiente montaje en la placa de prototipo:



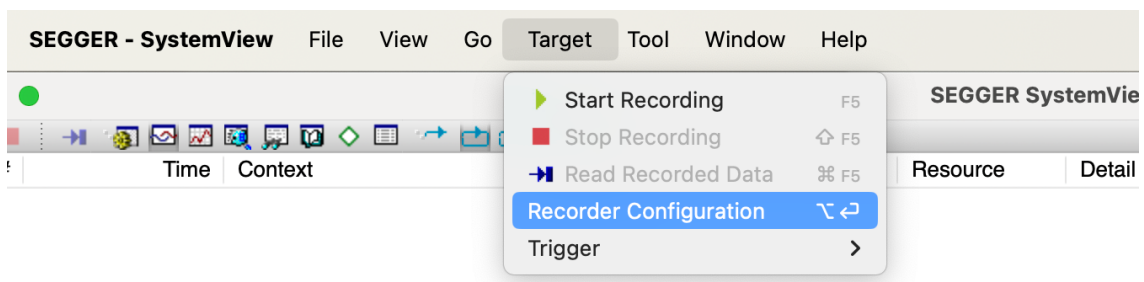
Para comunicar un PC actual con una UART lo hacemos a través de un puerto USB. Para ello necesitamos un bridge USB-UART, como el que integra el propio ESP32 para su UART0. En la caja de prácticas dispones de una placa de este tipo. El modelo de la placa no es exactamente el de la imagen, pero observa que tiene los mismos pines. La conexión es:

Bridge USB-UART	Placa ESP32
GND -----	GND
TX -----	RX2
RX -----	TX2

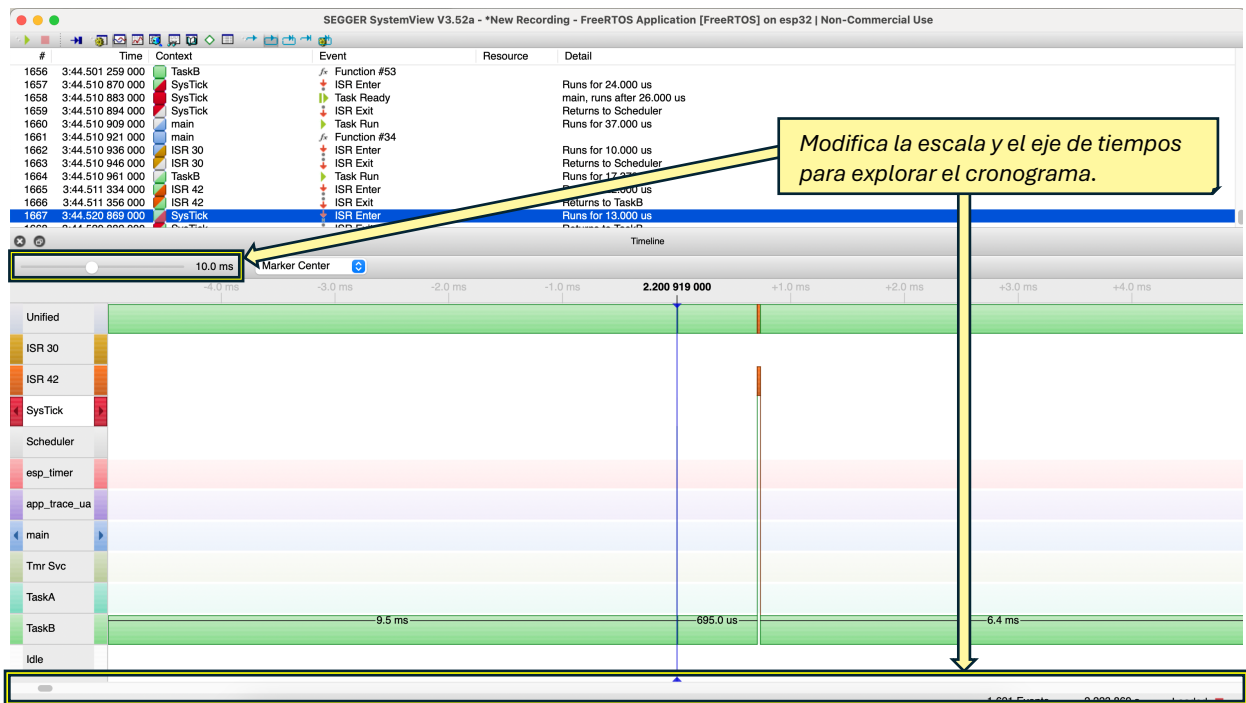
4. Antes de compilar el código, hay que hacer una modificación al código de la librería de Segger. En la primera línea de código de app_main, pulsa CTRL mientras haces click en la función SEGGER_SYSVIEW_Conf. Al hacerlo, se abrirá un desplegable con dos opciones de fichero fuente, selecciona la segunda para abrir el fichero SEGGER_SYSVIEW_config_FreRTOS.c. Busca la función _cbSendSystemDesc, comenta el código tal y como se aprecia en la siguiente imagen y guarda el fichero.

```
151  /*****
152  *
153  *      _cbSendSystemDesc()
154  *
155  *  Function description
156  *      Sends SystemView description strings.
157  */
158  static void _cbSendSystemDesc(void) {
159      char irq_str[32];
160      SEGGER_SYSVIEW_SendSysDesc("N=\"SYSVIEW_APP_NAME\",D=\"SYSVIEW_DEVICE_NAME\",C=\"SYSVIEW_CORE_NAME\",0=FreeRTOS");
161      snprintf(irq_str, sizeof(irq_str), "I#d=SysTick", SYSTICK_INTR_ID);
162      SEGGER_SYSVIEW_SendSysDesc(irq_str);
163      /*
164      size_t isr_count = sizeof(esp_isr_names)/sizeof(esp_isr_names[0]);
165      for (size_t i = 0; i < isr_count; ++i) {
166          if (esp_isr_names[i] == NULL || (ETS_INTERNAL_INTR_SOURCE_OFF + i) == SYSTICK_INTR_ID)
167              continue;
168          snprintf(irq_str, sizeof(irq_str), "I#d=%s", ETS_INTERNAL_INTR_SOURCE_OFF + i, esp_isr_names[i]);
169          SEGGER_SYSVIEW_SendSysDesc(irq_str);
170      }
171      */
172  }
```

5. Conecta el cable del ESP32 a un puerto USB, y utiliza el cable auxiliar para conectar el bridge USB-UART que hemos cableado a otro puerto USB de tu ordenador.
6. Compila y carga el código en la placa.
7. Abre la aplicación SystemView y configura el puerto USB que hemos conectado al bridge USB-UART mediante el menú Target -> Recording Configuration.



8. Asegúrate de tener el ESP funcionando (puedes comprobarlo con el monitor del IDE, como lo hemos hecho habitualmente, ya que el puerto es diferentes y no influirá en la captura de trazas), selecciona Start Recording, espera un instante (1 segundo es suficiente), y selecciona Stop Recording. El resultado debe parecerse a la siguiente imagen.



Apartado I: Aprendizaje mediante observación del planificador.

- Describe brevemente y con tus propias palabras qué hace el código proporcionado.
- Utiliza el cronograma de eventos de SystemView para hacer una medición precisa de los siguientes intervalos (modifica la escala temporal para ajustar el tamaño de los intervalos para que sea visible). Aporta capturas de pantalla.
 - Duración del procesamiento de la tarea A.
 - Duración del procesamiento de la tarea B.
 - Duración de la ejecución del Tick del sistema.
 - Duración del cambio de contexto (Scheduler)
 - Duración de la ISR del Timer que libera el semáforo para activar la tarea en cada iteración (Tmr Svc)
- Comenta la llamada a `ESP_LOGI` que hay dentro de la función `matrix_multiply`. Compila y carga de nuevo el firmware en la placa. Registra nuevamente un segundo aproximado de trazas. ¿Qué duración tiene ahora la tarea A?. ¿Qué buena práctica de programación a bajo nivel te sugiere este resultado?
- Sigue los pasos y responde a las preguntas:

Pasos:

- Lanza Menuconfig, navega a la opción Component Config -> ESP System Settings y deshabilita la opción *Initialice Task Watchdog Timer on startup*. Guarda la configuración. (No hagas esto en una aplicación en producción)
- Modifica la configuración de las tareas para que ambas multipliquen matrices de 50x50, 10 veces.
- Compila y carga el nuevo firmware.
- Captura trazas durante 5 segundos.

...

...

Preguntas:

- a. ¿Qué hace el planificador cada Tick del sistema? ¿Por qué?. Aporta captura de pantalla.

Pasos

- Establece la prioridad de la tarea A en 2, modificando el quinto argumento de la llamada a función `xTaskCreatePinnedToCore` de la tarea A.
- Compila y carga el nuevo firmware.
- Captura trazas durante 5 segundos.

Preguntas:

- b. ¿Qué hace el planificador cada Tick del sistema? ¿Por qué?. Explica las diferencias con respecto a la ejecución anterior. Aporta captura de pantalla.
- c. Los cambios de contexto los realiza la rutina Scheduler. Observa que hay dos tipos de invocaciones de esta rutina en las trazas. Ilustra con capturas de pantalla un ejemplo de cada tipo y trata de explicar qué lo provoca en cada caso.